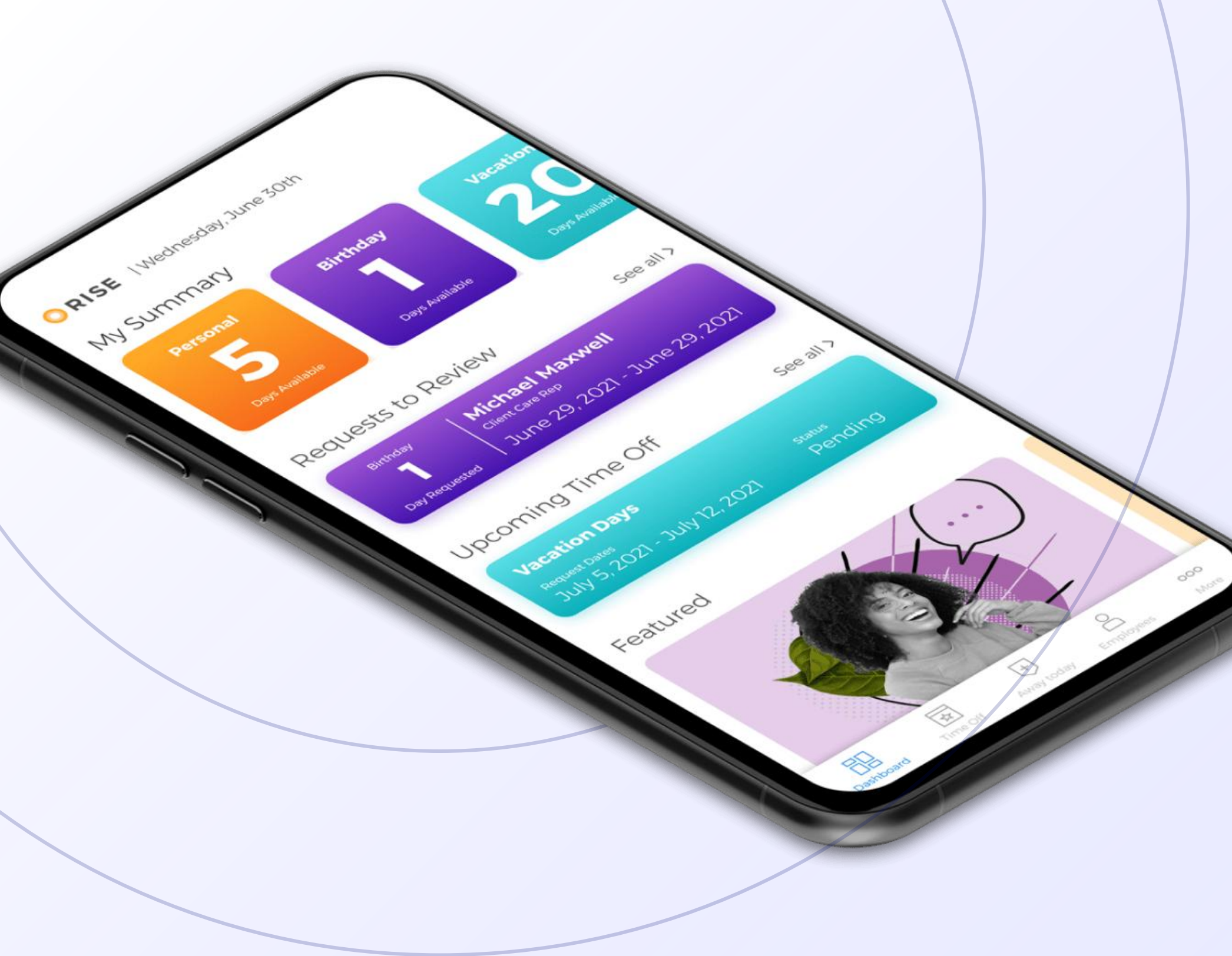




Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

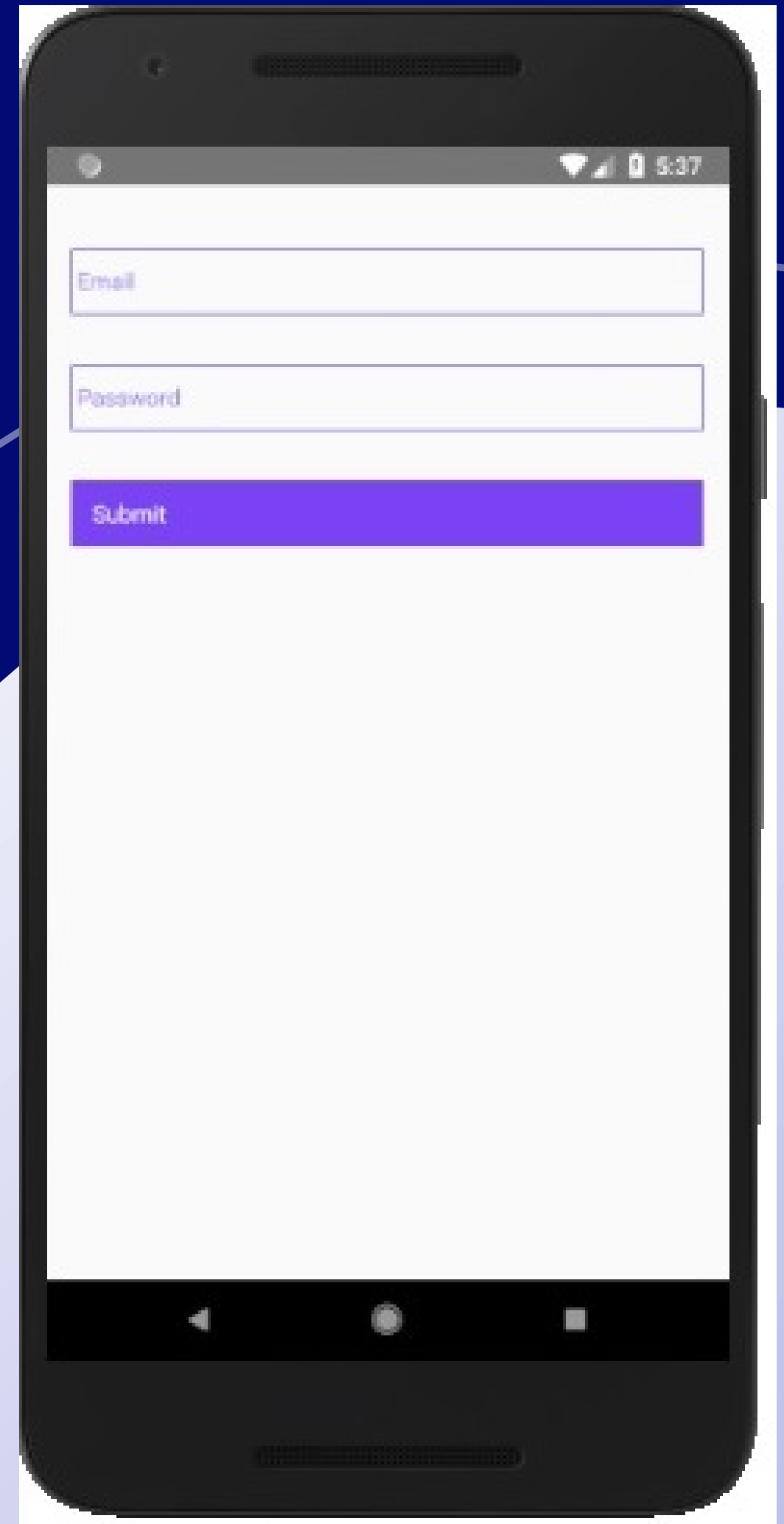
2026. január 8.

13. A

Tanárok: Dr. Koncz Adrienn és Simon Martin

TextInput komponens

- A **TextInput**, vagy más néven **szövegbeviteli mező**, a React Native egyik alapvető komponense, amelyet akkor használunk, amikor a felhasználónak szöveget kell beírnia az alkalmazásban.
- Ilyen lehet például egy név, egy üzenet, egy keresőkifejezés vagy akár egy jelszó megadása.



```
import React, { useState } from 'react';
import { Text, TextInput, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
```

```
export default function App() {
  const [text, setText] = useState('');

  return (
    <SafeAreaView style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Írj ide valamit..."
        value={text}
        onChangeText={setText}
      />
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#999',
    padding: 10,
    marginBottom: 10,
  },
});
```

Egyszerű példa

- A TextInput egy beviteli mezőt jelenít meg
- A placeholder egy segítő szöveg, ami addig látszik, amíg nem írunk bele
- A value az aktuálisan beírt szöveget tartalmazza
- Az onChangeText minden változáskor frissíti az állapotot

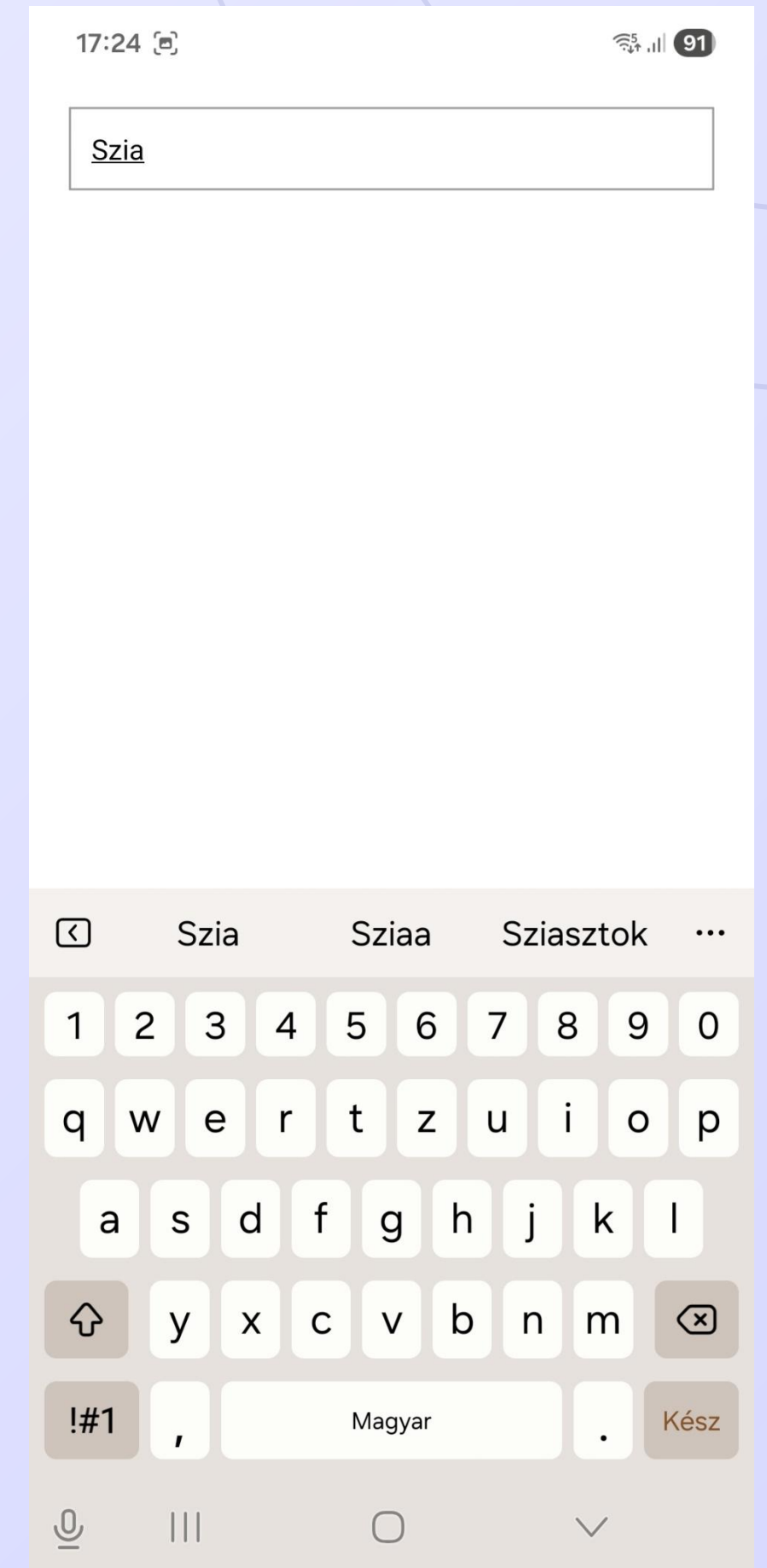
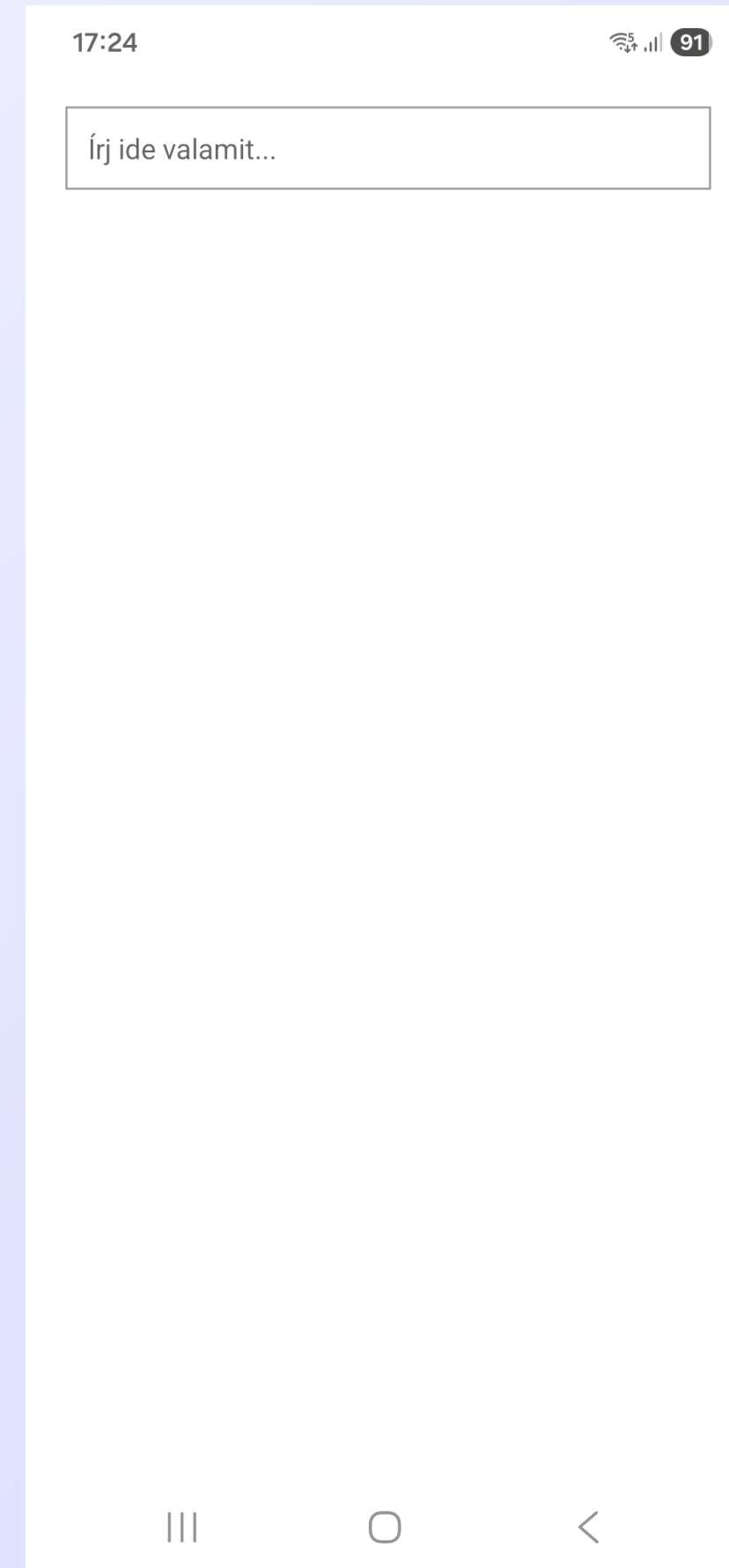
```
import React, { useState } from 'react';
import { Text, TextInput, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
```

```
export default function App() {
  const [text, setText] = useState('');

  return (
    <SafeAreaView style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Írj ide valamit..."
        value={text}
        onChangeText={setText}
      />
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#999',
    padding: 10,
    marginBottom: 10,
  },
});
```

Egyszerű példa



```

import React, { useState } from 'react';
import { Text, TextInput, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  const [text, setText] = useState('');

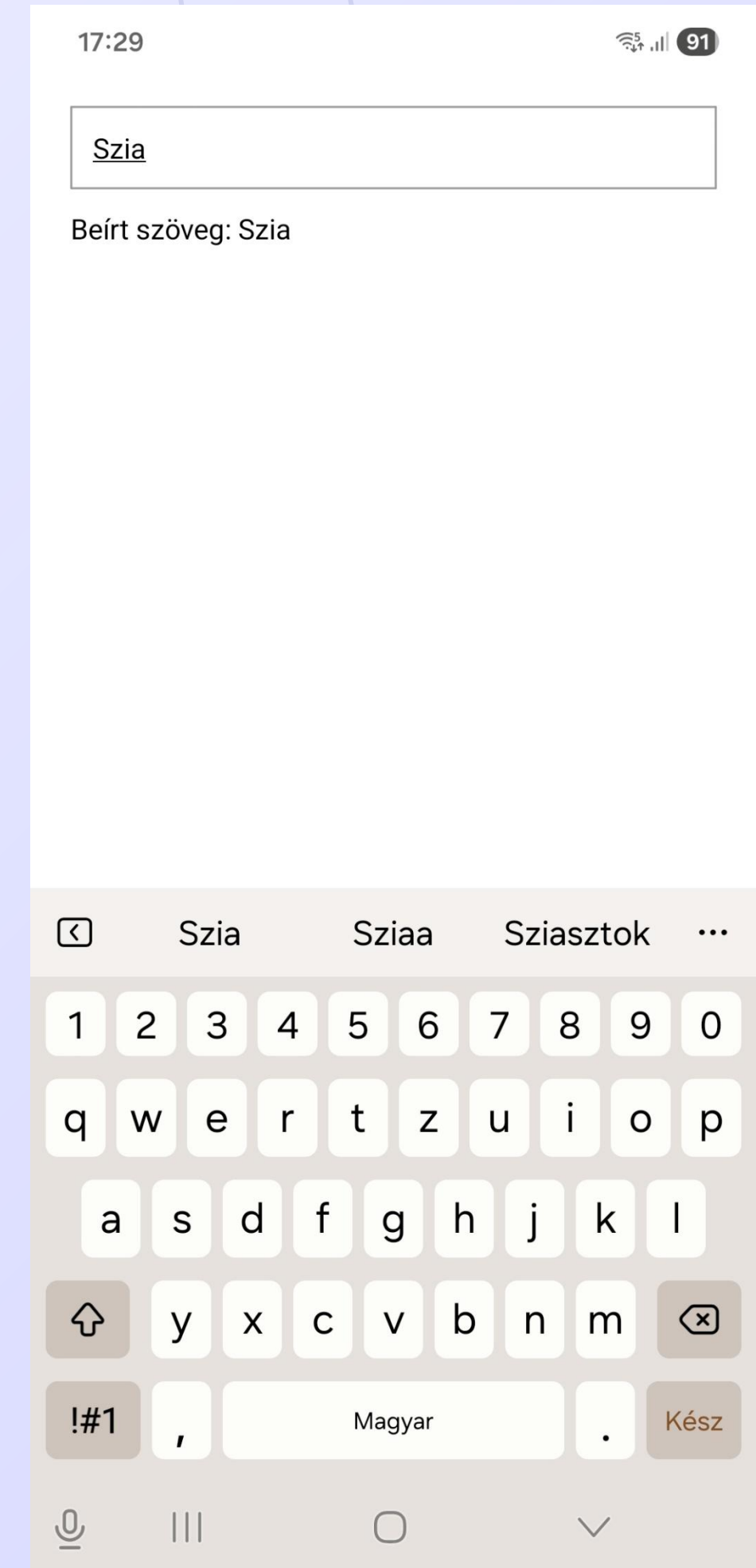
  return (
    <SafeAreaView style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Írj ide valamit..."
        value={text}
        onChangeText={setText}
      />
      <Text>Beírt szöveg: {text}</Text>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#999',
    padding: 10,
    marginBottom: 10,
  },
});

```

Egyszerű példa

- A beírt szöveg azonnal megjelenik a Text komponensben is.



TextInput komponens

- A komponens számos beállítási lehetőséget (propot) kínál, amelyekkel a szövegbevitel viselkedése testre szabható.
- Beállíthatjuk például, hogy:
 - történjen-e automatikus javítás,
 - legyen-e automatikus nagybetűsítés,
 - megjelenjen-e segítő szöveg a mezőben (placeholder),
 - vagy milyen típusú billentyűzet jelenjen meg (például numerikus billentyűzet).
- A leggyakoribb használat az, hogy elhelyezünk egy TextInput mezőt, és figyeljük, mit ír be a felhasználó.
- Ezt az **onChangeText** esemény segítségével tesszük meg, amely minden szövegváltozáskor lefut, és átadja az aktuálisan beírt szöveget az alkalmazásnak.

Mi az az onChangeText?

- Az onChangeText egy eseménykezelő, amely minden alkalommal meghívódik, amikor a TextInput tartalma megváltozik.
- Ez megtörténik például akkor, ha a felhasználó:
 - új karaktert ír be,
 - töröl egy karaktert,
 - bemásol szöveget,
 - vagy az automatikus javítás módosítja a beírt szöveget.
- Az onChangeText nem egy eseményobjektumot ad vissza, hanem közvetlenül magát a szöveget, mégpedig szöveg (string) formájában.
- Ez teszi különösen egyszerűvé a használatát, hiszen az aktuális tartalom azonnal elmenthető egy változóba vagy állapotba (state).

További események és metódusok a TextInput-hoz kapcsolódóan

- A TextInput más eseményeket is biztosít:
 - **onSubmitEditing** – akkor fut le, amikor a felhasználó elküldi a szöveget (például az Enter megnyomásával),
 - **onFocus** – akkor aktiválódik, amikor a beviteli mező fókuszbba kerül.
- A komponenshez két fontos metódus is tartozik:
 - **.focus()** – programozottan fókuszbba helyezi a TextInput mezőt,
 - **.blur()** – programozottan elveszi a fókuszt a mezőről.
- Ezeket akkor használjuk, amikor az alkalmazás logikája dönti el, mikor legyen aktív a beviteli mező.

További események és metódusok a TextInput-hoz kapcsolódóan – példa kód

```
import React, { useState } from 'react';
import { Text, TextInput, Button, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

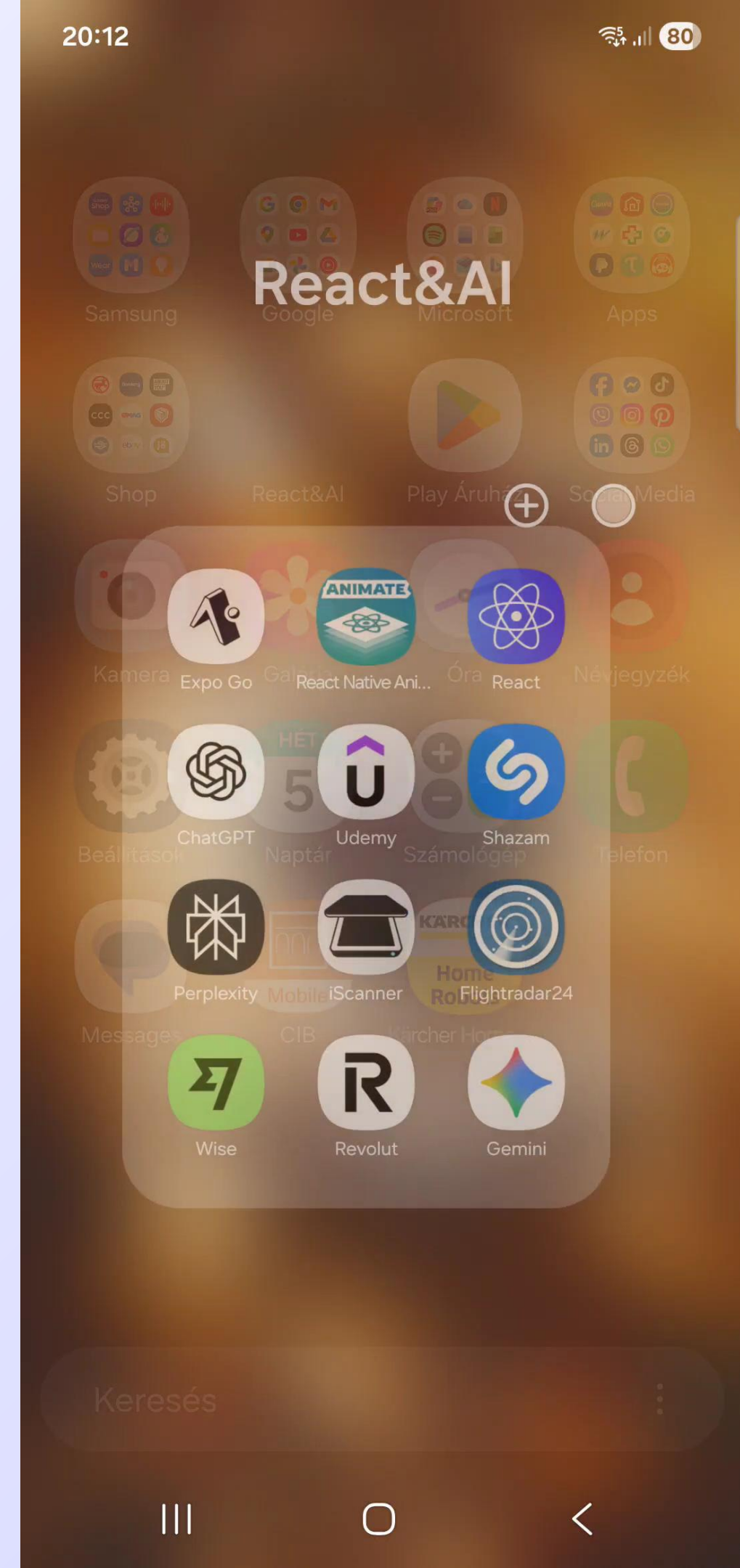
export default function App() {
  const [text, setText] = useState('');
  const [submittedText, setSubmittedText] = useState('');
  const [focused, setFocused] = useState(false);
  const [showInput, setShowInput] = useState(true);

  return (
    <SafeAreaView style={styles.container}>
      {showInput && (
        <TextInput style={[styles.input, { borderColor: focused ? 'green' : '#999' }], []}
          placeholder="Írj ide, majd Enter"
          autoFocus={false}
          onFocus={() => setFocused(true)}
          onChangeText={(value) => setText(value)}
          onSubmitEditing={() => {
            setSubmittedText(text);
            setShowInput(false);
            setFocused(false);
          }}
        />
      )}

      <Button title="Újra fókuszba helyez" onPress={() => {setText(''); setShowInput(true); }} />
      <Text style={styles.text}> Aktuális szöveg: {text} </Text>
      <Text style={styles.text}> Beküldött szöveg: {submittedText} </Text>
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    padding: 20,
  },
  input: {
    borderWidth: 2,
    padding: 10,
    marginBottom: 10,
  },
  text: {
    marginTop: 5,
  },
});
```

További események és metódusok a TextInput-hoz kapcsolódóan – példa kód működése



Többsoros TextInput mező

- Egyes beállítások csak akkor működnek, ha a multiline prop értéke egyértelműen true vagy false.
- Fontos tudni, hogy többsoros (multiline=true) beviteli mező esetén az egyoldali szegélystílusok (például alsó vagy bal oldali szegély) nem jelennek meg.
- Ha mégis ilyen hatást szeretnénk elérni, akkor a TextInput komponenst egy View komponensbe kell becsomagolni, és a stílust a View-ra kell alkalmazni.

Többsoros TextInput mező – példa kód

```
import React from 'react';
import { TextInput, StyleSheet } from 'react-native';
import { SafeAreaView, SafeAreaProvider } from 'react-native-safe-area-context';

const MultilineTextInputExample = () => {
  const [value, onChangeText] = React.useState('Multiline Placeholder Text');

  return (
    <SafeAreaProvider>
      <SafeAreaView
        style={{
          flex: 1,
          backgroundColor: value.toLowerCase(),
        }}>
        <TextInput
          multiline —————> engedélyezi a többsoros szövegbevitelt
          numberOfLines={4} —————> 4 soros lesz a TextInput mező DE! csak logikailag
          maxLength={500} —————> Ennyi karaktert lehet beleírni a mezőbe
          value={value}
          onChangeText={onChangeText}
          underlineColorAndroid="transparent" —————> Androidon alpból van egy alsó
            vonal a TextInput alatt. Ezzel
            kikapcsoljuk, hogy ne zavarja a
            saját keretet.
          style={styles.textInput}
        />
      </SafeAreaView>
    </SafeAreaProvider>
  );
};
```

```
const styles = StyleSheet.create({
  textInput: {
    borderWidth: 2,
    borderColor: '#2979FF',
    padding: 10,
    margin: 12,
    minHeight: 120, —————> Ténylegesen így
    textAlignVertical: 'top',      lesz többsoros
    backgroundColor: '#fff',
  },
});

export default MultilineTextInputExample;
```

Többsoros TextInput mező – példa kód működése

21:30

75

Multiline Placeholder Text



Fontosabb TextInput propok (az eddigieken kívül)

autoCapitalize: Szabályozza az automatikus nagybetűsítést. Nem minden billentyűzettípus támogatja.

Lehetséges értékek:

- characters – minden karakter nagybetűs,
- words – minden szó első betűje nagybetűs,
- sentences – minden mondat első betűje nagybetűs (alapértelmezett),
- none – nincs automatikus nagybetűsítés.

caretHidden: Ha igaz, a kurzor nem jelenik meg a beviteli mezőben.
Alapértelmezett értéke: false.

További props-ok:

<https://reactnative.dev/docs/textinput>

keyboardType paraméter – Milyen billentyűzet jelenjen meg?

A **keyboardType** segítségével megadhatjuk, hogy a TextInput mező fókuszbba kerülésekor milyen billentyűzet jelenjen meg a felhasználónak.

Célja: gyorsabb adatbevitel, kevesebb hibázás, jobb felhasználói élmény.

default: Az alapértelmezett billentyűzet jelenik meg.

- Betűk, számok, speciális karakterek.
- Általános szöveghez használjuk.

email-address: Email bevitelhez optimalizált billentyűzet.

- Tartalmazza az @ és . karaktereket.
- Nem nagybetűsít automatikusan.
- Tipikus használat: email mező

numeric: Csak számok jelennek meg.

- Előfordulhat, hogy . vagy , is megjelenik (platformfüggő).
- Tipikus használat: életkor, darabszám

number-pad: Kizárólag számjegyek (0–9) jelennek meg.

- Nincs tizedesjel.
- Tipikus használat: PIN kód, irányítószám

keyboardType paraméter – Milyen billentyűzet jelenjen meg?

decimal-pad: Számjegyek és tizedesjel jelenik meg.

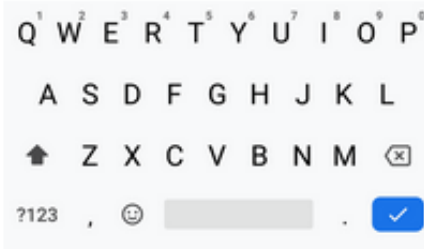
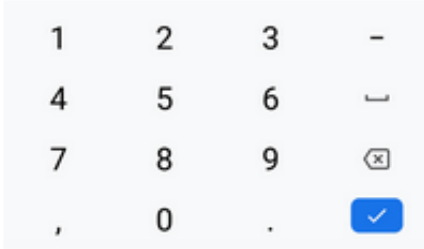
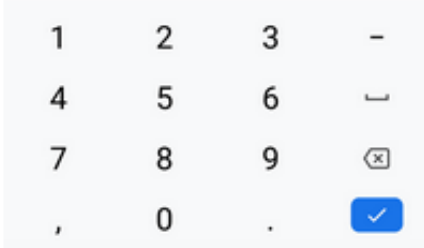
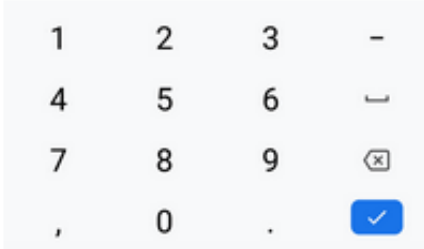
- Platformtól függően . vagy ,.
- Tipikus használat: ár, súly, mért érték

phone-pad: Telefonszám bevitelhez készült billentyűzet.

- Tartalmazza a +, *, # karaktereket is.
- Tipikus használat: telefonszám

url: Webcímekhez optimalizált billentyűzet.

- Gyakran tartalmaz .com, / gombokat.
- Tipikus használat: weboldal URL

keyboardType	iOS	Android
default		
number-pad		
decimal-pad		
numeric		
email-address		
phone-pad		

keyboardType paraméter – Milyen billentyűzet jelenjen meg?

ascii-capable



numbers-and-punctuation



url



name-phone-pad



initializes on 'default'

web-search



visible-password



with secureTextEntry flag

Példa feladatok

1. Bejelentkezés / Regisztrációs űrlap

23:11

71

Bejelentkezés

Email

pl.: example@email.com

Jelszó

Kurzor elrejtése (caretHidden)

Belépek

Nincs még fiókod? [Regisztráció](#)

III

O

<

23:11

71

Regisztráció

Teljes név

Pl.: Minta Név

Email

pl.: example@email.com

Jelszó

Kurzor elrejtése (caretHidden)

Regisztrálok

Már van fiókod? [Bejelentkezés](#)

III

O

<

Példa feladatok

2. To-Do lista extrákkal

